

PERSONAL EXPENSE TRACKER

PROJECT REPORT

Submitted by

Veer Singh Chauhan

Registration Number: 26BAI10805

Python Project – VITyarthi Build Your Own Project

1. Introduction

Personal Expense Tracker is a Python command-line application developed to record and manage daily expenses. It uses a simple menu-driven interface and stores records in a CSV file so that expense data remains available after the program is closed.

The project demonstrates Python functions, loops, conditional statements, exception handling, CSV file handling, dictionaries and lists, input validation, and date processing. The application provides practical operations for recording, viewing, searching, summarising, and deleting expenses.

2. Problem Statement

Managing small daily expenses manually can make it difficult to keep records organised and calculate spending patterns. A lightweight local application can provide a structured way to enter, store, retrieve, search, summarise, and delete expense records.

The project solves this problem using Python and CSV storage without requiring an external database or third-party packages.

3. Functional Requirements

ID	Function	Requirement
FR1	Add Expense	Accept date, category, description and amount and save the record.
FR2	View Expenses	Display all stored expense records.
FR3	Search Expenses	Search using description or category.
FR4	Category Summary	Calculate category-wise spending totals.
FR5	Total Spending	Show expense count, total spending and average expense.
FR6	Monthly Summary	Filter records by MM-YYYY and calculate the monthly total.
FR7	Delete Expense	Delete a record using its ID.
FR8	Validation	Validate date, amount, category, description, IDs and menu choices.
FR9	Persistent Storage	Create, read and update expenses.csv.

4. Non-functional Requirements

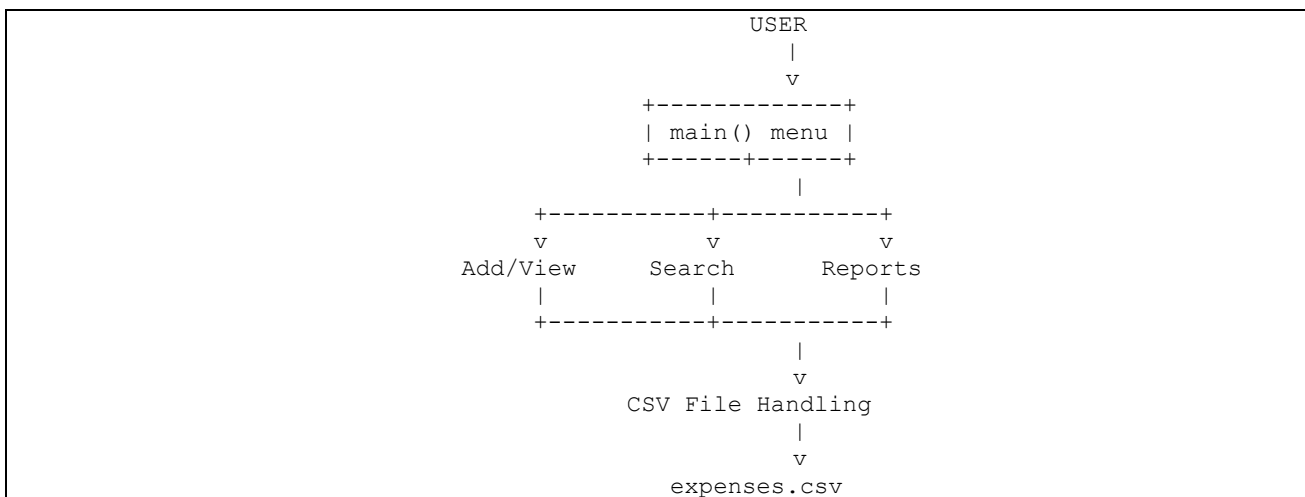
Requirement	Implementation
Usability	Simple menu-driven interface and clear prompts.
Reliability	Input validation and exception handling reduce common input errors.
Maintainability	Functions divide the program into logical operations.
Performance	CSV processing is appropriate for a small

	personal dataset.
Resource Efficiency	Uses local storage and only Python standard-library modules.
Portability	Runs on systems with Python 3.x.
Data Persistence	Records remain available in expenses.csv between runs.

5. System Architecture

The application uses a simple logical architecture. The user interacts with `main()` and the menu. The selected operation calls a feature function. Validation functions check input, while CSV functions load and save records. Reporting functions process the loaded records.

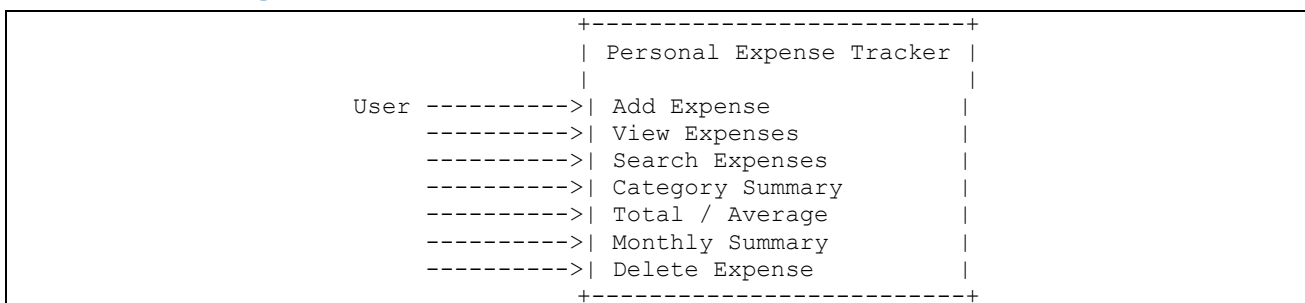
System Architecture



System Architecture

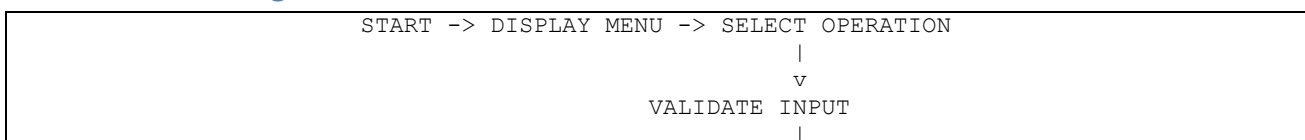
6. Design Diagrams

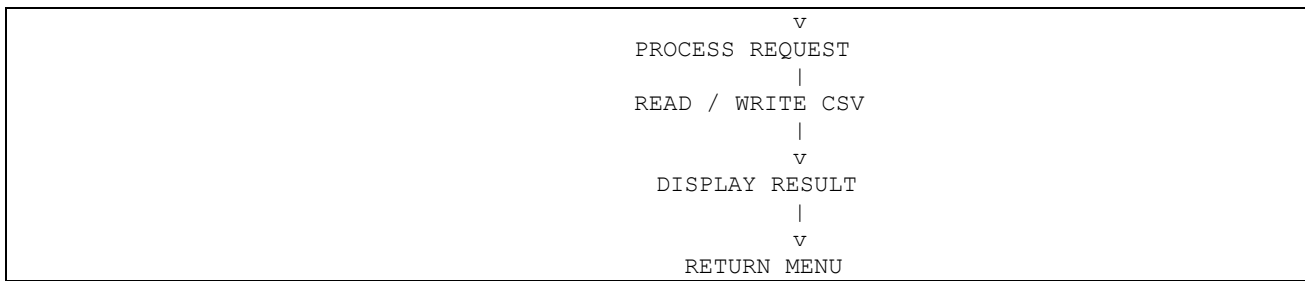
6.1 Use Case Diagram



6.1 Use Case Diagram

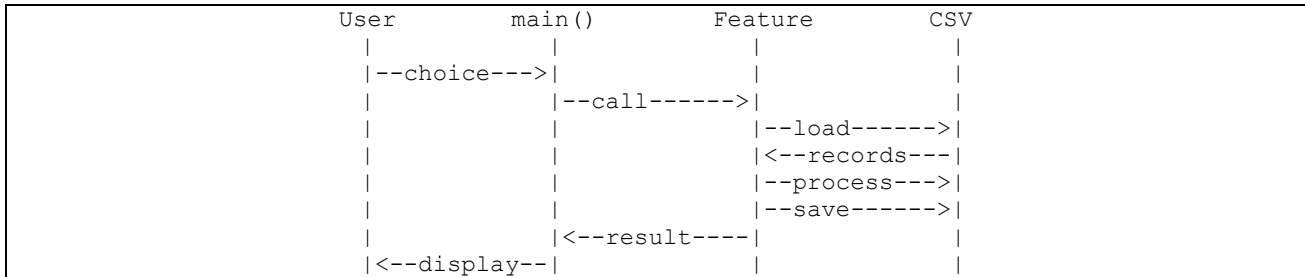
6.2 Workflow Diagram





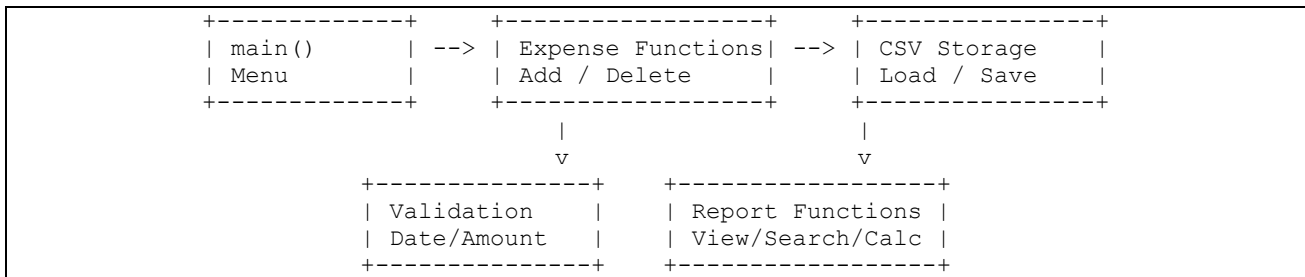
6.2 Workflow Diagram

6.3 Sequence Diagram



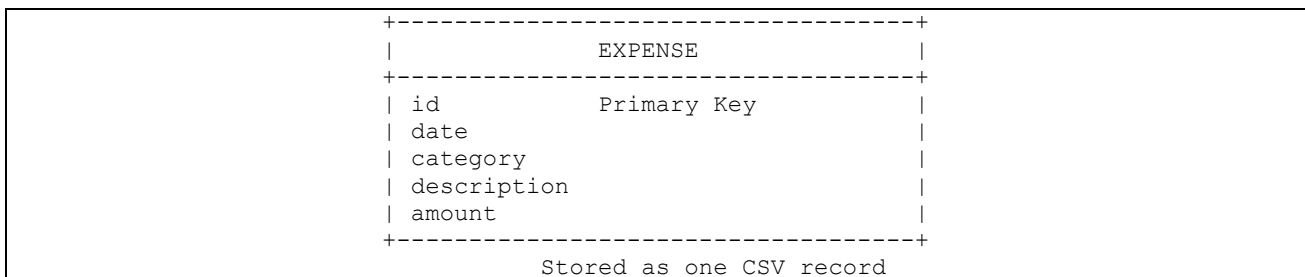
6.3 Sequence Diagram

6.4 Component Diagram



6.4 Component Diagram

6.5 ER / Storage Diagram



6.5 ER / Storage Diagram

Because the application uses a CSV file rather than a relational database, the storage diagram represents the structure of one Expense record. The id field uniquely identifies each record.

7. Design Decisions & Rationale

Decision	Rationale
CSV storage	Simple, persistent and requires no database installation.

Command-line interface	Keeps the project lightweight and focused on Python fundamentals.
Functions	Separates operations such as adding, searching, reporting and deletion.
Input validation	Prevents common invalid values and improves reliability.
Automatic IDs	Makes each expense easy to identify for deletion.
Standard library only	csv, os and datetime are sufficient for the current project.

8. Implementation Details

8.1 Data Storage

DATA_FILE points to expenses.csv. setup_file() creates the file and header when required. load_expenses() reads records with csv.DictReader(), and save_expenses() writes records using csv.DictWriter().

8.2 Expense Entry

add_expense() loads existing records, generates the next ID, collects the date, category, description and amount, appends the record and saves the updated list.

8.3 Validation

get_amount() accepts only positive numeric amounts. get_date() validates DD-MM-YYYY. choose_category() restricts the selection to the available categories. Empty descriptions are rejected and invalid menu choices are handled.

8.4 Search and Reports

search_expenses() performs a case-insensitive search in description and category. category_summary() calculates category totals. total_spending() calculates total and average spending. monthly_summary() filters dates by MM-YYYY.

8.5 Deletion

delete_expense() accepts an expense ID, removes the matching record from the list and writes the updated records back to the CSV file.

8.6 Main Menu

main() creates the storage file if necessary and repeatedly displays the eight menu options until the user selects Exit.

9. Screenshots / Results

The supplied program was reviewed against its implemented operations. Representative sample results are shown below. For final portal submission, actual screenshots from the user's own terminal run may be added if required.

Operation	Representative Result
Add Expense	Expense added successfully with a generated ID.
View Expenses	Records displayed with ID, date, category,

	description and amount.
Category Summary	Category totals and overall total displayed.
Total Spending	Count, total spending and average expense displayed.
Monthly Summary	Records matching the requested MM-YYYY displayed with monthly total.
Search	Matching description/category records displayed.
Delete	Selected expense removed successfully.

9.1 Sample Data

id	date	category	description	amount
1	25-09-2026	Food	Lunch	120.00
2	25-09-2026	Travel	Bus fare	40.00
3	26-09-2026	Education	Notebook	80.00

10. Testing Approach

Testing focuses on functional behaviour, input validation, file persistence and complete menu flow. The following test cases can be executed directly from the command line.

Test	Input / Condition	Expected Result
Add valid expense	Valid date, category, description and positive amount	Record saved.
Invalid amount	Zero, negative or non-numeric amount	Error message and retry.
Invalid date	Incorrect date format	Error message and retry.
Invalid category	Number outside available categories	Error message and retry.
View	Existing CSV records	Records displayed.
Search	Keyword such as Food	Matching records displayed.
Category summary	Several categories	Category totals displayed.
Total spending	Several records	Count, total and average displayed.
Monthly summary	Valid MM-YYYY	Matching records and total displayed.
Delete	Existing ID	Record removed.
Invalid delete	Missing/non-numeric ID	Error message displayed.
Exit	Option 8	Program terminates.

11. Challenges Faced

- CSV values are stored as strings, so IDs and amounts must be converted when performing calculations.
- User input can be invalid, so validation loops and try-except blocks were required.
- Monthly summaries require converting stored date strings into datetime values.
- Deleting a record requires updating and rewriting the CSV data.
- Keeping the interface simple while providing several useful operations required careful menu design.

12. Learnings & Key Takeaways

- Learned practical CSV file handling using Python.
- Improved understanding of functions, loops and conditional statements.
- Practised exception handling and user-input validation.
- Learned to process lists of dictionary records and calculate summaries.
- Understood how multiple functions can form a complete menu-driven application.
- Learned the importance of testing both valid and invalid inputs.
- Practised documenting system architecture, workflow and storage design.

13. Future Enhancements

- Add an edit/update expense option.
- Add budgets and budget alerts.
- Add graphical charts for spending analysis.
- Add Excel or PDF export.
- Move to SQLite for larger datasets.
- Add user accounts or password protection.
- Add date-range and amount-range filters.
- Develop a GUI or web interface.
- Add automated unit tests for individual functions.

14. References

1. Python Documentation – csv module.
2. Python Documentation – datetime module.
3. Python Documentation – os module.
4. VITyarthi – Build Your Own Project, General Project Instructions & Submission Guidelines.
5. Personal Expense Tracker source code developed by Veer Singh Chauhan.

15. Project Information

Item	Details
Project Title	Personal Expense Tracker
Student Name	Veer Singh Chauhan
Registration Number	26BAI10805
Application Type	Command-line application
Programming Language	Python 3.x
Storage	expenses.csv
External Packages	None
Main Source File	expense_tracker.py

Report prepared according to the project-report requirements provided in the VITyarthi Build Your Own Project guidelines.